

Übersicht:

Python: Interpretiert. Verbreitet für: AI/ML development, data science, automation, back-end, scripting.

Backend: FastAPI / Flask / Django

JavaScript: Interpretiert/JIT-kompiliert. Verbreitet für: web development, front-end, back-end, APIs, full-stack development.

Frontend: React / Vue / Angular. Backend: Node.js / Express / NestJS

TypeScript: JavaScript mit statischer Typisierung. Wird zu JavaScript transpiliert. Verbreitet für: web development, large-scale front-end, back-end, APIs.

Frontend: Angular / React. Backend: Node.js / NestJS

PHP: Interpretiert/JIT-fähig. Verbreitet für: web development, back-end, APIs, CMS, server-side applications.

Backend: Laravel / Symfony / Native PHP

Java: Kompiliert zu JVM-Bytecode. Verbreitet für: enterprise applications, back-end, APIs, business software, Android.

Backend: Spring / Spring Boot

Scripting / Admin:

PowerShell: Interpretiert. .NET-basiert. Verbreitet für: Windows administration, Active Directory, automation, scripting, security administration.

Umgebung: Windows / AD / .NET

Bash: Interpretiert. Verbreitet für: Linux/Unix administration, automation, scripting, DevOps, security automation.

Umgebung: Linux / Unix

System / Low level:

C: Kompiliert zu native machine code. Verbreitet für: operating systems, embedded systems, system software, low-level programming.

Security: Memory / pointers / buffers / stack & heap / malloc / free

Compiler: Im Voraus übersetzt (Compile-Zeit). Erzeugt eine ausführbare Datei (z. B. .exe). Bricht bei Syntaxfehlern den gesamten Prozess ab, bevor das Programm startet.

Vorteil: läuft viel schneller.

Interpreter: Übersetzt und ausgeführt direkt zur Laufzeit (Run-Time). Stoppt die Ausführung erst, wenn er auf einen Fehler in einer Zeile stößt.

Vorteil: Vor jedem Test eine neue Übersetzung nicht benötigt.

Gutes Buch: /Users/Said/Desktop/Unterricht/Depo/Bücher/OWASP-ASVS-2025

Topic	Python	JavaScript	TypeScript	PHP	Java	PowerShell	Bash	C
Variable	x = 5	let x = 5;	let x: number = 5;	\$x = 5;	int x = 5;	\$x = 5	x=5	int x = 5;
String	s = "Hello"	let s = "Hello";	let s: string = "Hello";	\$s = "Hello";	String s = "Hello";	\$s = "Hello"	s="Hello"	char s[] = "Hello";
Boolean	True / False	true / false	true / false	true / false	true / false	\$true / \$false	exit status 0 = success/true	bool x = true; uses <stdbool.h>)
Null	None	null / undefined	null / undefined	null	null	\$null	no null value;	NULL / null pointer
Feld des aktuellen Objekts	self.name	this.name	this.name	\$this->name	this.name	\$this.Name / \$user["Name"]	--	obj.name / ptr->name
Array / List	[1, 2, 3]	[1, 2, 3]	const arr: number[] = [1,2,3]	\$arr = [1,2,3];	int[] arr = {1,2,3};	\$arr = @(1,2,3)	arr=(1 2 3)	int arr[] = {1,2,3};
Access	arr[0]	arr[0]	arr[0]	\$arr[0]	arr[0]	\$arr[0]	\${arr[0]}	arr[0]
Length	len(arr)	arr.length	arr.length	count(\$arr)	arr.length	\$arr.Count	\${#arr[@]}	sizeof(arr) / sizeof(arr[0])
Add / Append	arr.append(4)	arr.push(4)	arr.push(4)	\$arr[] = 4;	fixed array; list.add(4)	\$arr += 4	arr+=(4)	fixed size; no append
Remove last	arr.pop()	arr.pop()	arr.pop()	array_pop(\$arr)	list.remove(...)	\$arr = \$arr[0..(\$arr.Count-2)]	unset 'arr[-1]'	manual
Remove by index	arr.pop(1)	arr.splice(1,1)	arr.splice(1,1)	unset(\$arr[1])	list.remove(1)	filter / rebuild	unset 'arr[1]'	manual
Contains	x in arr	arr.includes(x)	arr.includes(x)	in_array(\$x, \$arr)	list.contains(x)	\$arr -contains \$x	loop / [[...]]	loop
Iterate	for x in arr:	for (const x of arr)	for (const x of arr)	foreach (\$arr as \$x)	for (int x : arr)	foreach (\$x in \$arr)	for x in "\${arr[@]}"	for (...)
Slice	arr[1:3]	arr.slice(1,3)	arr.slice(1,3)	array_slice(\$arr,1,2)	Arrays.copyOfRange(arr,1,3)	\$arr[1..2]	\${arr[@]:1:2}	manual / pointer arithmetic
First element	arr[0]	arr[0]	arr[0]	\$arr[0]	arr[0]	\$arr[0]	\${arr[0]}	arr[0]
Last element	arr[-1]	arr[arr.length-1]	arr[arr.length-1]	\$arr[count(\$arr)-1]	arr[arr.length-1]	\$arr[-1]	\${arr[-1]}	arr[n-1]
Constant	UPPER_CASE convention	const MAX = 5;	const MAX = 5;	const MAX = 5;	static final int MAX = 5;	Set-Variable MAX -Option Constant	readonly MAX=5	const int MAX = 5;
Type conversion	int(x)	Number(x)	Number(x) / x as Type	(int)\$x	Integer.parseInt(x)	[int]\$x	\$((x))	strtol(...)

Inline if/else	a if cond else b	cond ? a : b	cond ? a : b	cond ? \$a : \$b	cond ? a : b	\$cond ? \$a : \$b (PS 7+)	\$((cond ? a : b))	cond ? a : b
use x if it exists, otherwise use y	x if x is not None else y	x ?? y	x ?? y	\$x ?? \$y	Objects.requireNonNullElse(x,y)	\$x ?? \$y	\${x:-default}	x ? x : y / explicit check
obj exists → obj.x obj null/undefined → undefined	getattr(obj,'x',None) / explicit check	obj?.x	obj?.x	\$obj?->x	Optional / explicit null check	\$obj?.x (PS 7+)	--	explicit null-pointer check
*list → values unpack **dict → key=value unpack	*args / **kwargs bsp : def add(a, b): return a + b numbers = [7, 9] add(*numbers)	...arr / {...obj}	...arr / {...obj}	...\$arr	Type... args / collections	@params splatting	\${arr[@]} / "\$@"	variadic ... / manual copy
String concatenation	a + b	a + b	a + b	\$a . \$b	a + b	\$a + \$b	\$a\$b	snprintf(...) / strcat(...)
Diff-Type in String	f"Hi {name}"	`Hi \${name}`	`Hi \${name}`	Hi \$name	Hi + name	Hi \$name	Hi \$name	snprintf(...)
Function	def sum(x):	function sum(x) { }	function sum(x: number): number { }	function sum(\$x) { }	static int sum(int x) { }	function Get-Sum { param(\$x) }	sum() { }	int sum(int x) { }
Return	return x	return x;	return x;	return \$x;	return x;	return \$x	return 0 (exit)	return x;
Lambda (anonym function)	lambda x: x + 1	x => x + 1	(x: number) => x + 1	fn(\$x) => \$x + 1	x -> x + 1	{ param(\$x) \$x + 1 }	--	function pointer / callback
Callback (function to run later)	bsp : def say_hi(): print("Hi") def ryo(callback): print("Before") callback() print("After") ryo(say_hi) Output: Before Hello After	func(callback)	func(callback)	func(\$callback)	Consumer / Function / listener	script block / delegate	function name / command	function pointer
If	if a < b:	if (a < b) { }	if (a < b) { }	if (\$a < \$b) { }	if (a < b) { }	if (\$a -lt \$b) { }	if ["\$a" -lt "\$b"]; then	if (a < b) { }
Else if	elif ...:	else if (...) { }	else if (...) { }	elseif (...) { }	else if (...) { }	elseif (...) { }	elif ...; then	else if (...) { }
Else	else:	else { }	else { }	else { }	else { }	else { }	else	else { }
Switch / match	bsp : match x:	switch (x) { }	switch (x) { }	match (\$x) /	switch (x) { }	switch (\$x) { }	case "\$x" in	switch (x) { }

(multi-branch condition)	case 1: print("one") case 2: print("two")			switch				
For loop	for i in range(5):	for (let i=0; i<5; i++)	for (let i=0; i<5; i++)	for (\$i=0; \$i<5; \$i++)	for (int i=0; i<5; i++)	for (\$i=0; \$i -lt 5; \$i++)	for ((i=0; i<5; i++))	for (int i=0; i<5; i++)
For each	for x in arr:	for (const x of arr)	for (const x of arr)	foreach (\$arr as \$x)	for (int x : arr)	foreach (\$x in \$arr)	for x in "\${arr[@]}"	usually index/pointer loop
While	while x < 5:	while (x < 5) { }	while (x < 5) { }	while (\$x < 5) { }	while (x < 5) { }	while (\$x -lt 5) { }	while ["\$x" -lt 5]; do	while (x < 5) { }
AND	and	&&	&&	&&	&&	-and	&&	&&
OR	or					-or		
NOT	not	!	!	!	!	-not	!	!
Equality	==	===	===	===	String: .equals() Others: ==	-eq	strings: [["\$a" == "\$b"]] ; integers: -eq	String: strcmp(a, b) == 0 Others: ==
Not equal	!=	!==	!==	!==	!=	-ne	!=	!=
Less / Greater	< > <= >=	< > <= >=	< > <= >=	< > <= >=	< > <= >=	-lt -gt -le -ge	-lt -gt -le -ge	< > <= >=
Modulo	%	%	%	%	%	%	\$(x % y)	%
Increment	x += 1	x++	x++	\$x++	x++	\$x++	((x++))	x++
Try / Catch (exception handling)	try / except	try / catch	try / catch	try / catch	try / catch	try / catch	no native exceptions	no standard exceptions
Throw / Error	raise ValueError(...)	throw new Error(...)	throw new Error(...)	throw new Exception(...)	throw new Exception(...)	throw "error"	exit 1 / return status	return error code / errno
Exception suppression (ignore/hide)	except: pass	empty catch / ignored Promise rejection	empty catch / ignored Promise rejection	@func(...) / empty catch	empty catch	-ErrorAction SilentlyContinue	2>/dev/null / true	ignored return value / errno
Single comment	# comment	// comment	// comment	// comment	// comment	# comment	# comment	// comment
Block comment	—	/* ... */	/* ... */	/* ... */	/* ... */	<# ... #>	--	/* ... */
Async function await: event loop	bsp : async def get_ip():	async function f() { }	async function f(): Promise<void> { }	no async; library/framework-	no async; CompletableFutur	no async; Jobs / .NET Task	background process/job	library/thread/event-loop-specific

can keep working	ip = await dns() return ip			specific	e / executors / virtual threads	patterns		
Await (wait inside async flow)	await f()	await f()	await f()	library/framework- specific	no await keyword; Future.get()/join() / reactive patterns	no await keyword; Wait-Job / .NET Task waiting	wait	pthread_join() / library-specific
Promise / Future (result of async work)	asyncio.Task / Future	Promise	Promise<T>	Promise/Future library-specific	CompletableFutur e<T> / Future<T>	Job / System.Threading .Tasks.Task	PID / background job	pthread / future library
Import / module load	import os	import ... from '...' / require(...)	import ... from '...' / dynamic import(...)	require/include: FILE loading use: name import	import package.Class	Import-Module ...	source file.sh / . file.sh	#include <...> (preprocessor include)
Runtime / platform (where code executes)	CPython / Python runtime	Browser / Node.js	JS runtime after transpilation often Node.js/browser	PHP runtime / PHP-FPM	JVM	PowerShell + .NET	Unix shell	native executable
Web framework (HTTP app)	FastAPI / Flask / Django	Express / Fastify	NestJS / Express	Laravel / Symfony	Spring Boot	--	--	library/server-spec fic
App / framework initialization (e.g. express())	app = FastAPI() / app = Flask(__name__)	const app = express()	await NestFactory.creat e(AppModule) / express()	framework bootstrap (Laravel/Symfon manages it)	SpringApplication. run(App.class, args)	--	--	server/library-spec fic
Package manager (install/update dependencies)	pip / uv / poetry	npm / yarn / pnpm	npm / yarn / pnpm	Composer	Maven / Gradle	PowerShellGet / PSResourceGet	apt/dnf/brew + shell tools	system package manager / build system
Declare dependencies	requirements.txt / pyproject.toml	package.json	package.json	composer.json	pom.xml / build.gradle	module manifest .psd1	project/tool-specifi c	Makefile / CMakeLists.txt / pkg config
Dependency lock file (pins resolved versions)	uv.lock / poetry.lock / requirements pinning	package-lock.json / yarn.lock / pnpm-lock.yaml	package-lock.json / yarn.lock / pnpm-lock.yaml	composer.lock	Gradle/Maven locking or dependency management	module/version pinning	tool-specific lockfile	tool/build-system-s pecific
Framework supplies dependencies	FastAPI Depends / framework DI	framework/contai ner-specific	NestJS constructor DI	Laravel/Symfony container	Spring @Autowired / constructor DI	PowerShell/.NET patterns	--	manual / library-specific
Decorator (metadata/wrappe r on	@decorator	decorators (supported syntax) /	@Controller / @Injectable / decorators	attributes #[...]	@GetMapping / @PreAuthorize / annotations	attributes [ValidateSet()] / .NET attributes	--	attributes/compiler extensions

class/function)		framework-specif.						
Middleware / filter (runs before/after request handler)	FastAPI dependency / Django middleware	app.use(...)	NestJS middleware / Guard / Interceptor	Laravel/Symfony middleware	Spring Filter / Interceptor / Security filter	pipeline/function wrapper patterns	shell wrapper/function	server/library-specific
Request handler (receives routed request)	route function / class-based view	(req, res) => { ... }	@Controller() class / handler method	Controller method	@RestController / controller method	server/framework-specific	server/framework-specific	server/library-specific
Service layer (business logic)	UserService / service module	userService / service module	@Injectable() UserService	service class	@Service class	function / .NET service object	function/script module	function/module
Repository / DAO (DB access layer)	repository / SQLAlchemy session	data-access module / ORM client	Repository<T> / Prisma client	Eloquent/Doctrine repository pattern	JpaRepository / DAO	DB helper / .NET data access	DB CLI/helper function	DB client/library wrapper
DTO (structured request data object)	Pydantic BaseModel / dataclass	plain object + validation schema	class CreateUserDto / interface	FormRequest / DTO class	record/class CreateUserRequest	[pscustomobject] / class	--	struct
Endpoint (URL mapp to handler)	@app.get('/users/{id}')	app.get('/users/:id', ...)	@Get('users/:id')	Route::get('/users/{id}', ...)	@GetMapping('/users/{id}')	--	--	server/library-specific
HTTP method (GET/POST/PUT)	request.method / decorator	req.method / app.get post	@Get / @Post / req.method	\$request->method() / Route::post	@GetMapping / @PostMapping	server/framework-specific	server/framework-specific	server/library-specific
Query parameter (e.g. ?id=123)	request.query_params.get('id') / request.args.get('id')	req.query.id	@Query('id') id	\$request->query('id')	@RequestParam String id	--	--	library-specific
Path parameter (e.g. /users/{id})	path param / request.view_args	req.params.id	@Param('id') id	route parameter / \$id	@PathVariable String id	--	--	library-specific
Request body (JSON/form data) — user-input	await request.json() / request.get_json()	req.body	@Body() body	\$request->input(..) / \$request->all()	@RequestBody Dto body	--	--	library-specific
HTTP header may be user-input	request.headers.get('X')	req.headers['x']	@Headers('x') / req.headers	\$request->header('X')	@RequestHeader('X')	server/framework-specific	server/framework-specific	server/library-specific
Cookie client controlled state	request.cookies.get('session')	req.cookies.session	req.cookies.session	\$request->cookie('session') / \$_COOKIE	@CookieValue('session')	server/framework-specific	server/framework-specific	server/library-specific
Response body	return data /	res.send() /	res.send() /	return	ResponseEntity /	server/framework-specific	server/framework-specific	server/library-specific

(data to client)	Response(...)	res.json()	res.json()	response()->json(...)	return object	specific	specific	fic
Response status (HTTP code)	Response(..., status_code=...)	res.status(403)	res.status(403)	response(..., 403)	ResponseEntity.status(...)	server/framework-specific	server/framework-specific	server/library-specific
Response header	response.headers['X']=...	res.set('X', value)	res.set('X', value)	response()->header('X', value)	HttpHeaders / response.setHeader	server/framework-specific	server/framework-specific	server/library-specific
Redirect (send client to URL)	redirect(url)	res.redirect(url)	res.redirect(url)	redirect(\$url)	sendRedirect / redirect view	server/framework-specific	server/framework-specific	server/library-specific
Console / stdin input / user-input	input('Name: ')	Browser: prompt(); Node: readline	Browser: prompt(); Node: readline	readline() / STDIN	Scanner.nextLine()	Read-Host 'Name'	read name	fgets(..., stdin)
CLI arguments	sys.argv	process.argv	process.argv	\$argv	args	\$args	\$1 / \$@	argv[1]
Environment var (external runtime input/config)	os.getenv('NAME')	process.env.NAME	process.env.NAME	getenv('NAME')	System.getenv('NAME')	\$env:NAME	\$NAME	getenv('NAME')
Input validation (allowlist/type/range/format checks)	Pydantic / validators / allowlist checks	Zod/Joi/framework validation	class-validator / DTO / Zod	\$request->validate(...) / Validator	Bean Validation @Valid	ValidateSet / ValidatePattern / manual	case/regex/allowlist checks	manual/library validation
Mass assignment (request fields → object fields automatically)	Model(**data) / schema-to-model mapping	Object.assign(model, req.body) / spread	DTO/object spread / ORM create(data)	Model::create(\$request->all())	@RequestBody -> entity / BeanUtils	splatting / property assignment	--	struct field assignment
URL parsing (normalize URL)	urlib.parse / httpx.URL	new URL(...)	new URL(...)	parse_url(...)	URI / URL	[System.Uri]	url tools / manual	library-specific
Path construction (build filesystem path)	os.path.join / pathlib.Path	path.join(...)	path.join(...)	DIRECTORY_SEPARATOR / Path utilities	Path.of(...)	Join-Path	\$dir/\$file	snprintf(...) / library
Path normalise (resolve path)	Path.resolve() / os.path.realpath()	path.resolve() / path.normalize()	path.resolve() / path.normalize()	realpath()	Path.normalize() / toRealPath()	Resolve-Path	realpath / readlink -f	realpath()
File read (filesystem sink)	open(...) / Path.read_text()	fs.readFile()	fs.readFile()	file_get_contents()	Files.readString()	Get-Content	cat / read	fopen() / fgets()
File write (filesystem sink)	open(...,'w') / Path.write_text()	fs.writeFile()	fs.writeFile()	file_put_contents()	Files.writeString()	Set-Content / Out-File	> file / printf	fopen(...,'w') / fprintf()

File delete (filesystem sink)	os.remove() / Path.unlink()	fs.unlink()	fs.unlink()	unlink()	Files.delete()	Remove-Item	rm -- "\$path"	remove()
Temporary file (secure temp-file creation)	tempfile.NamedTemporaryFile()	fs.mkdtemp() / os.tmpdir()+safe lib	fs.mkdtemp() / safe lib	tempnam() / tmpfile()	Files.createTemp File()	New-TemporaryFil e	mktemp / mktemp -d	mkstemp()
File upload (untrusted file input)	UploadFile / request.files	req.file / req.files (e.g. multer)	@UploadedFile()	\$_FILES / \$request->file()	MultipartFile	web-framework-s pecific	server/tool-specifi c	library-specific
File permission / ownership	os.chmod()	fs.chmod()	fs.chmod()	chmod()	Files.setPosixFile Permissions()	Get-Acl / Set-Acl	chmod / chown	chmod() / chown()
Archive extraction (review paths / Zip Slip risk)	zipfile.extractall() / tarfile	archive library extract	archive library extract	ZipArchive	ZipInputStream / library	Expand-Archive	unzip / tar	libarchive / library
Dynamic load (code/file at runtime)	importlib.import_ module(name)	require(path) / import(path)	import(path)	include(\$file) / require(\$file)	Class.forName(na me) / classloader	Import-Module \$path / . \$path	source "\$file"	dlopen(path)
Choose method/type by name at runtime	getattr(obj, name) / getattr(...)()	obj[name]() / Reflect.*	obj[name]() / Reflect.*	call_user_func() / variable functions	Class.forName / Method.invoke	& \$command / Invoke-Command	indirect expansion / eval / command name	dlsym() / function pointers
Dynamic code exec. (string/data → executable code; high risk)	eval() / exec()	eval() / Function(...)	eval() / Function(...)	eval()	scripting engine / dynamic compilation only	Invoke-Expressio n	eval / bash -c / sh -c	no built-in eval; system/compiler/li brary
Process creation (start child process)	subprocess.Pope n() / run()	child_process.spa wn()	child_process.spa wn()	proc_open()	ProcessBuilder	Start-Process	command & / exec	fork() / exec*()
Shell / OS execution (command-execut ion sink)	subprocess.run(... , shell=True)	child_process.exe c() / execSync()	child_process.exe c() / execSync()	shell_exec() / system() / exec()	Runtime.exec() / ProcessBuilder + shell	Invoke-Expressio n / cmd.exe / powershell -Command	eval / sh -c / bash -c / unquoted expansion	system() / popen() / exec*()
Command arg handling (structured args vs shell string)	subprocess.run([c md,arg], shell=False)	spawn(cmd,[arg], {shell:false})	spawn(cmd,[arg], {shell:false})	proc_open() / escaped arg APIs	new ProcessBuilder(c md,arg)	Start-Process -ArgumentList (review quoting)	arrays + quoted "\$var"	execve(path, argv, envp)
Privilege change / elevation	os.setuid() / setgid()	process.setuid() / setgid()	process.setuid() / setgid()	posix_setuid() / setgid()	OS/JVM-specific	Start-Process -Verb RunAs /	sudo / su / setpriv	setuid() / setgid()

						token APIs		
Outbound HTTP request (SSRF sensitive sink)	requests / httpx	fetch / axios	fetch / axios / HttpService	cURL / Guzzle	HttpClient	Invoke-WebRequest / Invoke-RestMethod	curl / wget	libcurl / HTTP library
TLS verification bypass (certificate verification disabled)	verify=False	rejectUnauthorized:false / custom agent	rejectUnauthorized:false / custom agent	CURLOPT_SSL_VERIFYPEER=false	trust-all TrustManager / disabled hostname verification	-SkipCertificateCheck	curl -k / wget --no-check-certificate	CURLOPT_SSL_VERIFYPEER=0
Network listener (open server socket/port)	socket.bind()/listen() / server	server.listen() / net.createServer()	server.listen() / framework	stream_socket_server() / framework	ServerSocket / embedded server	TcpListener / Start-Job server patterns	nc -l / socat	socket() / bind() / listen()
Bind address (where the server listens for connections)	0.0.0.0 / host=...	0.0.0.0 / host arg	0.0.0.0 / host arg	0.0.0.0 / config	server.address / 0.0.0.0	IPAddress.Any	0.0.0.0	INADDR_ANY
DB connection / client	DB-API / SQLAlchemy engine	driver/pool (pg/mysql2/etc.)	driver/pool / ORM DataSource	PDO / mysqli / framework DB	DataSource / JDBC	SqlConnection / Invoke-Sqlcmd	psql/mysql/sqlite CLI	DB client library
ORM (Object Relational Map: DB tables ↔ objects/classes)	SQLAlchemy / Django ORM	Prisma / Sequelize	Prisma / TypeORM	Eloquent / Doctrine	JPA / Hibernate	--	--	--
ORM query (query DB through ORM API)	User.query.filter(...)/ session.query(...)	prisma.user.findMany(...)	repository.find(...)/ prisma.*	User::where(...)-> get()	repository.findBy.. / Criteria	--	--	--
ORM raw query (raw SQL through ORM, injection risk)	text(...) / raw()	\$queryRaw / sequelize.query()	repository.query() / \$queryRaw	DB::select() / whereRaw()	nativeQuery=true / createNativeQuery()	--	--	--
Raw SQL (manually built SQL, injection risk)	cursor.execute('SELECT '+input)	db.query('SELECT '+input)	db.query('SELECT '+input)	PDO->query('SELECT '.\$input)	Statement.executeQuery('SELECT '+input)	Invoke-Sqlcmd -Query \$query	mysql/psql -e "\$query"	DB-specific query string
Parameterized	cursor.execute('...	driver-specific	driver/ORM-specific	PDO	PreparedStatement	SqlCommand.Parameters	DB-client/API-specific	prepared

SQL (SQL + bound parameters)	WHERE id=%s',(id,))	placeholders + bound values	ic placeholders + bound values	prepare/execute / DB::select('...?',[id])	nt	ameters / parameterized provider	cific; shell quoting is NOT SQL parameterization	statements in DB library
SQL transaction (commit / rollback / atomic unit)	commit()/rollback()/context manager	transaction API / BEGIN-COMMIT	transaction API / @Transactional-style ORM	DB::transaction() / PDO transaction	@Transactional / Connection commit	SqlTransaction	BEGIN/COMMIT via DB client	DB library transaction APIs
Template engine (render HTML/text from templates)	Jinja2 / Django templates	EJS / Pug / framework-specific	Angular templates / Handlebars	Blade / Twig	Thymeleaf / JSP	--	--	--
Escaped HTML output (framework template encodes special character)	{{ value }} (default escaping in common templates)	React {value} / template-engine escaping	Angular {{ value }}	Blade {{ \$value }}	Thymeleaf th:text	--	--	--
Raw / unsafe HTML output (escaping bypass, XSS-risk)	Jinja safe / Markup	innerHTML / dangerouslySetInnerHTML	bypassSecurityTrustHtml() / direct innerHTML	Blade {!! \$value !!}	th:utext / raw writer	--	--	--
JSON parse (text → data structure)	json.loads()	JSON.parse()	JSON.parse()	json_decode(..., true)	ObjectMapper.readValue()	ConvertFrom-Json	jq / tool-specific	cJSON / library
XML parsing	ElementTree / lxml	DOMParser / XML library	XML library	SimpleXML / DOMDocument	DocumentBuilder / SAX / JAXB	.NET XML classes	xmlint / xmlstarlet	libxml2 / library
Deserialization (data → object; risky with untrusted input)	pickle.loads(data) / yaml.load(data)	library-specific object deserialization	library-specific object deserialization	unserialize()	ObjectInputStream.readObject()	Import-Clixml / BinaryFormatter-like .NET APIs (review trust boundary)	tool/library-specific	library-specific
Safer Deserialization	yaml.safe_load(data)							
Base64 encoding (not encryption)	base64.b64encode()	Buffer.from(...).toString('base64')	Buffer.from(...).toString('base64')	base64_encode()	Base64.getEncoder()	[Convert]::ToBase64String()	base64	library-specific
Regex (pattern matching)	re.compile()/re.search()	RegExp / /.../	RegExp / /.../	preg_match()	Pattern / Matcher	-match / [regex]	[[... =~ ...]] / grep -E	regcomp()/regexexec()
Authentication (who are you?)	framework auth / current_user / dependency	auth middleware / req.user	NestJS Guard / req.user	auth middleware / auth()->user()	Spring Security / SecurityContext	WindowsIdentity / AD / credential APIs	OS user / PAM/tool-specific	application-specific

Authorization (what are you allowed to do?)	permission/role checks / decorator	authorization middleware / permission check	Guards / @Roles / policy	Gate / Policy / authorize()	@PreAuthorize / roles / ACL checks	AD group / ACL / token checks	UID/GID / sudoers / file permissions	application-specific
Session (server-side/login state)	framework session / session[...]	req.session	req.session / session middleware	session() / \$_SESSION	HttpSession / Spring Session	server/framework-specific	server/framework-specific	server/library-specific
JWT decode (read token claims; NOT verification)	jwt.decode(..., options={"verify_signature": False}) / library decode	jwt.decode(...)	jwt.decode(...)	JWT library decode	JWT parser/decode	JwtSecurityTokenHandler.ReadJwtToken()	base64/jq often used only to inspect	library-specific
JWT verify (validate signature + relevant claims)	jwt.decode(..., key, algorithms=[...]) / library verify	jwt.verify(...)	jwt.verify(...) / guard	JWT library verify	JWT verifier / Spring Security resource server	TokenValidationParameters / ValidateToken	tool/library-specific	library-specific
Authorization header (often Bearer token)	request.headers.get('Authorization')	req.headers.authorization	@Headers('authorization') / req.headers	\$request->header('Authorization')	@RequestHeader('Authorization')	\$Headers['Authorization']	curl -H 'Authorization: ...'	library-specific
Cookie set (server creates/updates cookie)	response.set_cookie(...)	res.cookie(...)	res.cookie(...)	setcookie() / response cookie	ResponseCookie / servlet cookie	server/framework-specific	server/framework-specific	server/library-specific
Secure cookie flags (Secure / HttpOnly / SameSite)	secure=True, httpOnly=True, sameSite=...	secure:true, httpOnly:true, sameSite:...	secure:true, httpOnly:true, sameSite:...	secure/httpOnly/sameSite options	Secure / HttpOnly / SameSite	server/framework-specific	server/framework-specific	server/library-specific
CSRF protection (protect state-changing browser requests)	Django CSRF / framework middleware	framework/middleware/token pattern	NestJS/framework middleware/token pattern	Laravel/Symfony CSRF	Spring Security CSRF	server/framework-specific	server/framework-specific	server/library-specific
CORS (browser cross-origin access policy)	CORS middleware / framework config	cors middleware / headers	enableCors()/middleware	framework CORS config	@CrossOrigin / Spring config	server/framework-specific	server/framework-specific	server/library-specific
Rate limiting (limit request frequency)	framework/middleware / reverse proxy	express-rate-limit / gateway	@nestjs/throttler / gateway	RateLimiter middleware	Bucket4j/gateway/framework	server/gateway-specific	server/gateway-specific	server/library-specific

Security headers (e.g. CSP / HSTS / X-Content-Type-Options)	framework middleware / proxy	helmet / res.setHeader	helmet / framework config	framework middleware / headers	Spring Security headers	server/framework-specific	server/framework-specific	server/library-specific
Secure random (cryptographic randomness for secrets/tokens)	secrets.*	crypto.randomBytes() / crypto.getRandomValues() / randomUUID()	crypto.randomBytes() / crypto.getRandomValues() / randomUUID()	random_bytes()/random_int()	SecureRandom	[System.Security.Cryptography.RandomNumberGenerator]	/dev/urandom / openssl rand	OS/library CSPRNG
Weak random (not for secrets/tokens)	random.random()	Math.random()	Math.random()	rand()/mt_rand()	java.util.Random (for secrets)	[System.Random]::new() / System.Random	\$RANDOM	rand()
Hashing (one-way digest; not enc)	hashlib.sha256()	crypto.createHash('sha256')	crypto.createHash('sha256')	hash('sha256', ...)	MessageDigest SHA-256	SHA256.Create() / Get-FileHash	sha256sum / openssl dgst	EVP/SHA library
Weak hash (e.g. MD5/SHA-1)	hashlib.md5()/sha1()	crypto.createHash('md5'/sha1')	crypto.createHash('md5'/sha1')	md5()/sha1()	MessageDigest MD5/SHA-1	MD5.Create()/SHA1.Create()	md5sum / sha1sum	MD5/SHA1 APIs
Password hashing (slow KDF for password storage)	argon2/bcrypt/scrypt library	argon2 / bcrypt / scrypt	argon2 / bcrypt / scrypt	password_hash()	PasswordEncoder / bcrypt / Argon2	ASP.NET/.NET password hasher or PBKDF2/Argon2 library	tool/library-specific	Argon2/bcrypt/PBKDF2 library
Password verify (compare password with stored hash)	argon2/bcrypt verify	argon2.verify()/bcrypt.compare()	argon2.verify()/bcrypt.compare()	password_verify()	PasswordEncoder .matches()	PasswordHasher.VerifyHashedPassword / library	tool/library-specific	library-specific
crypto (confident/integrity)	cryptography / PyNaCl	node:crypto / Web Crypto	node:crypto / Web Crypto	OpenSSL / sodium	JCA/JCE	System.Security.Cryptography	openssl / gpg / age	OpenSSL/libsodium
Hardcoded secret (secret in source)	api_key = "secret"	const apiKey = "secret"	const apiKey: string = "secret"	\$apiKey = "secret";	String apiKey = "secret";	\$apiKey = "secret"	API_KEY="secret"	char *api_key = "secret";
Environment secret (loaded from environment)	os.getenv('API_KEY')	process.env.API_KEY	process.env.API_KEY	getenv('API_KEY')	System.getenv('API_KEY')	\$env:API_KEY	\$API_KEY	getenv('API_KEY')
Secret manager (external secret storage)	keyring / cloud secret SDK	cloud secret SDK / vault client	cloud secret SDK / vault client	vault/cloud secret SDK	Vault/cloud secret SDK / keystore	SecretManagement / Credential Manager	secret CLI / vault agent / env injection	OS keyring / vault client library

Config file (application settings; may contain secrets)	.env / yaml / toml / settings	dotenv / config files	dotenv / config files	.env / config/*.php	application.properties / application.yml	psd1 / json / xml / env	env file / config	ini/json/custom config
Debug mode (dev/verbose behavior; review production exposure)	debug=True / FLASK_DEBUG	NODE_ENV=development / debug tooling	source maps/dev mode/framework config	APP_DEBUG / display_errors	dev profile / debug config	\$DebugPreference / verbose settings	set -x	debug build / symbols / verbose logging
Logging (record events; avoid password/token)	logging.*	console.* / logger library	Logger / console.*	Log::* / error_log()	SLF4J/Logger	Write-Information / Write-Verbose / logger	logger / echo	syslog() / fprintf()
Concurrency (work happening in parallel / overlapping)	threading / multiprocessing / asyncio	Promise / Worker Threads	Promise / Worker Threads	process/library-specific	Thread / ExecutorService / virtual threads	Start-Job / ForEach-Object -Parallel / .NET Task	& / wait / xargs -P / GNU parallel	pthread / fork
Lock / synchron (protect shared state / avoid race)	threading.Lock()/asyncio.Lock()	Atoms/Mutex library	Atoms/Mutex library	flock()/library	synchronized / Lock	Mutex / Monitor / Semaphore	flock / mkdir lock pattern	pthread_mutex / semaphore

Python = Befehle = Natürliche English gibi çok: .write .exists .remove <https://docs.python.org/3/library/> /Users/Said/Desktop/Python/Übung.py meine Notizen.

Module = Programmbausteine und müssen explizit importiert werden.

import sys: Interact with the Python runtime/system: read command-line arguments. exit a script. Example:

print(sys.argv) sys.argv → Liste von (alle) Argumenten sys.argv[0] → Name des Scripts sys.argv[1] → 1. Argument

stderr stores error messages

```
sys.stderr.write("Error occurred\n")
```

stdin reading user input:

```
for line in sys.stdin:  
    print(line)
```

stdout program output: can be redirected to files or pipes.

```
sys.stdout.write("Hello\n")
```

sys.exit()

import os: Interact with the operating system: work with files & folders. read environment variables.

os.system("clear") clear() **print(os.getcwd())** prints scripts path **os.mkdir('...')** **os.remove('...')**
os.chdir('/Users/Said/Desktop') (macOS) = (Windows): \ yerine \\ yap: 'C:\\Users\\Said' change directory

```
file_path = os.path.join("content", "images", "test.png")  
print(file_path)
```

Farklı OSLara uygun pathi otomatik oluşturur. sonuç: content/images/test.png

import subprocess: Run external commands: execute shell commands. run system tools. capture command output.

.run = .call Python → yeni process başlatır → komutu çalıştırır → bitmesini bekler → sonra Python devam eder.

```
result = subprocess.run(["ls", "-la"], capture_output=True, text=True) print(result.stdout)
```

import math: Mathematical operations & values (e, pi). Common use cases: square roots. rounding calculations. constants like pi. **print(math.sqrt(25))**

print(math.pi)

import random: Generate random values: random numbers. random choices.

```
print(random.randint(0, 90, 10)) 0/10/20.../90dan biri random. print(random.choice(["admin", "user", "guest"]))
```

from datetime import date: Work with dates and time: timestamps. log times. timenow = **datetime.now()**

import json: Work with JSON data. Python-Objekte in das JSON-Format umzuwandeln (Serialisierung) und wieder zurück.

Common use cases: read/write JSON files. parse API responses.

```
data = {"user": "admin", "active": True}
```

```
print(json.dumps(data, indent=4))
```

from urllib.parse import urljoin: URLs zusammenfügen/encoden/decoden/ zerlegen, einfache HTTP-Funktionen. urljoin() ile relative linkleri tam URL'ye çevirir.

`from bs4 import BeautifulSoup`: HTML içeriğini parse etmek ve içinden tag/link bulmak için kullanılır.

`import urllib3`: HTTP GET/POST Requests, Connection Pooling, Timeouts und Retries HTTPS-Verbindungen
`urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)`

`import requests`: Send to Website HTTP requests.

`import hashlib`: Für Hashes wie MD5, SHA256.

`import base64`: Für Base64-Encoding/Decoding, oft bei Tokens, Payloads, Basic Auth usw.

`import socket`: Für Netzwerk-/Port-/Low-Level-Verbindungen.

Some concepts:

● might become insecure ● error-prone / potentially less efficient ● safer / preferred pattern ● neutral Python syntax / concept

with:

```
f = open("file.txt")
data = f.read()
f.close()
```

 "close" forgettable, so safer version =>

```
with open("file.txt") as f:
    data = f.read()
```

 → Path Traversal weiterhin zu prüfen.

yield:

```
def get_numbers():
    return [1, 2, 3]
```

 lower mem-usage =>

```
def get_numbers():
    yield 1
    yield 2
    yield 3
```

```
numbers = get_numbers()
next(numbers) # 1
next(numbers) # 2
next(numbers) # 3
```

 . Statt 3-Werte im Speicher => Werte einzeln produziert, wenn gebraucht.

List comprehension:

```
admins = []
for u in users:
    if u.is_admin:
        admins.append(u)
```

 ⇔

```
admins = [u for u in users if u.is_admin]
```

 => [u for u in users if condition] = for-loop + if + append

Dictionary comprehension:

```
users = {u.id: u for u in user_list}
```

`def func(*args)`: → arbitrary positional arguments.

`def func(**kwargs)`: → arbitrary named arguments. `create_user(**request.json)` ⇔ mass assignment / unsafe object construction.

decorator: `@app.get(...)`, Security checks: `@login_required`, `@admin_required`.

alias: `import subprocess as sp` => `sp.run(...)`.

```
import plugins.pdf
import plugins.csv
import plugins.excel

if format == "pdf":
    module = plugins.pdf
elif format == "csv":
    module = plugins.csv
elif format == "excel":
    module = plugins.excel
```

```
import importlib
module = importlib.import_module(f"plugins.{format}")
```

Dynamic import: lower mem-usage=> but input-validation is a must!

Package structure: user input → routes.py → services.py → models.py

```
project/
├── app/
│   ├── __init__.py → package initialization import ...
│   ├── routes.py → HTTP endpoints @app.route("/users/<id>") def get_user(id): ...
│   ├── models.py → DB models class User: id username password_hash is_admin
│   └── services.py → business logic def delete_user(user_id): ...
├── tests/
├── requirements.txt
└── pyproject.toml
```

```
subprocess.run(
    "ping " + hostname,
    shell=True
)
```

```
subprocess.run(
    ["ping", hostname]
)
```

subprocess: **os.system():** Shell execution.

Backends:

Flask: → minimal → flexible → you assemble the components. Security → depends more on chosen extensions/configuration.

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/users")
def users():
    return jsonify({"users": []})
```

```
request.form
request.json
request.files
request.headers
request.cookies
request.view_args
```

response:

```
return render_template("user.html", user=user)
```

Manual HTML:

</> Python

```
return "<h1>" + name + "</h1>"
```

FastAPI: → API-focused → type hints + validation → async-friendly. Security → input validation, auth dependencies, async logic

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/users")
```

```
async def users():
```

```
    return {"users": []}
```

```
class UserInput(BaseModel):
```

```
    username: str
```

```
    email: str
```

```
@app.post("/users")
```

```
def create_user(user: UserInput):
```

```
    ...
```

Django: → complete web framework → many features included. Security → permissions, middleware, ORM queries, auth/session configuration.

JavaScript:

Variablen:

const name = "Ali"; const = nicht neu zugewiesen. name = "Said"; // Fehler

let count = 0; let = kann geändert werden.

Objekte:

```
const user = {  
  name: "Alice"  
};
```

```
user.name = "Bob"; // erlaubt
```

Nur `user = ...` wäre verboten.

Zugriff: `user.name = user["name"]`

Methoden:

```
// Klassische Funktion  
function add(a, b) {  
  return a + b;  
}
```

```
// Arrow Function  
const add = (a, b) => {  
  return a + b;  
};
```

```
// Kurzform  
const add = (a, b) => a + b;
```

Wichtige Unterschiede:

- `function` → eigenes `this`, hoisted, kann mit `new` genutzt werden
- `=>` → kein eigenes `this`, nicht hoisted, kein `new`

hoisted = Funktionsaufruf vor Funktionsdefinition möglich.

```
function User(name) {  
  this.name = name;  
}
```

```
const u = new User("Alice");
```

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

Funktionen können als Konstruktoren verwendet werden:

↔

Destructuring:

```
const user = {  
  name: "Alice",  
  age: 30  
};  
  
const { name, age } = user;  
name // Alice  
age  // 30
```

Web-Code:

```
app.get("/user", (req, res) => {  
  const id = req.query.id;  
  
  res.send("User: " + id);  
});
```

req.params.*
req.query.*
req.body.*
→ potentiell untrusted input

Falsy: Manche Werte automatisch als false: false, 0, "", null, undefined, NaN

```
if (username) {  
  login(username);  
}
```

Wenn username einen „wahren“ Wert hat.

Optional Chaining ?: `user?.profile?.name` bedeutet: Greife nur weiter zu, wenn der vorherige Wert nicht null/undefined ist.

Default mit ?? / ||:

```
const name = req.query.name ?? "guest";  
if (name === null || name === undefined) {  
  name = "guest";  
}
```

Bedeutet:

`x || "guest"` nimmt "guest" bei allen falsy-Werten.

async / await:

```
app.get("/user", async (req, res) => {  
  const user = await db.find(req.query.id);  
  res.json(user);  
});
```

await = Warte auf das Ergebnis dieser asynchronen Operation. async → erlaubt await.

Imports:

```
import fs from "fs";
```

```
const fs = require("fs");
```

```
fs.readFileSync(...)
```

Modern: `import express from "express";` Node/CommonJS:

```
const express = require("express");
```

Dann: `express(...)`

`const app = express();` erzeugt die Web-Anwendung / App-Instanz.

Review:

```
const { exec } = require("child_process");

app.get("/backup", async (req, res) => {
  const file = req.query.file ?? "default.txt";

  if (req.user?.isAdmin) {
    exec(`cp ${file} /backup/`);
    res.send("done");
  }
});
```

1. Zeile: Aus dem Modul `child_process` die Funktion `exec` herausgeholt.

TypeScript:

Variablen:

```
let age: number = 30;
let name: string = "Alice";
let active: boolean = true;
```

Funktionen:

```
function add(a: number, b: number): number {
  return a + b;
}
```

Arrow Function:

```
const add = (a: number, b: number): number => a + b;
```

Arrays:

```
const ids: number[] = [1, 2, 3];
const names: string[] = ["Alice", "Bob"];
```

⇔ `const ids: Array<number> = [1, 2, 3];`

Object Types:

```
const user: {  
  name: string;  
  age: number;  
} = {  
  name: "Alice",  
  age: 30  
};
```

↔

```
type User = {  
  name: string;  
  age: number;  
};  
  
const user: User = {  
  name: "Alice",  
  age: 30  
};
```

interface:

```
interface User {  
  id: number;  
  name: string;  
  admin: boolean;  
}
```

Dann:

```
const user: User = {  
  id: 1,  
  name: "Alice",  
  admin: false  
};
```

interface User → beschreibt, welche Felder ein User haben soll

Optional ?:

```
interface User {  
  name: string;  
  email?: string;  
}
```

Union Types |:

```
let id: string | number;
```

id darf string oder integer sein.

```
function findUser(id: string | number) {  
  // ...  
}
```

```
let result: User | null;
```

any / unknown:

```
let data: any;
```

any = TypeScript prüft mich hier kaum noch. Möglich:

```
data.foo.bar();  
data.execute();
```

```
let data: unknown;
```

unknown = sicherer Alternative. Vor Nutzung prüfen:

```
if (typeof data === "string") {  
  console.log(data.toUpperCase());  
}
```

PHP:

Variablen: beginnen mit \$

```
$x = 5;
```

```
$name = "Alice";
```

```
$isAdmin = true;
```

Verkettung:

```
$name = "Alice";
```

```
$msg = "Hello " . $name; oder $msg = "Hello $name";
```

Arrays:

```
$arr = [10, 20, 30];
```

```
echo $arr[0];
```

Associative Arrays: → Key gehört zu Value

```
$user = [  
  "name" => "Alice",  
  "admin" => false  
];
```

```
echo $user["name"];
```

```
];
```

```
echo $user["name"];
```

Funktionen:

```
function add($a, $b) {  
  return $a + $b;  
}
```

```
function add(int $a, int $b): int {  
  return $a + $b;  
}
```

Mit Typen: : int → Return-Typ

User Input:

URL: /user?id=123

Code: \$id = \$_GET["id"];

POST-Daten: \$username = \$_POST["username"];

Für Secure Review: \$_GET, \$_POST, \$_COOKIE, \$_FILES,...

Includes und Dateien:

include "config.php"; → Fehler, Script kann weiterlaufen

require "config.php"; → fataler Fehler, Script stoppt

Review:

```
<?php
$file = $_GET["file"];
$content = file_get_contents(
    "/var/uploads/" . $file
);
echo $content;
```

Java:

Variablen und Typen:

```
int age = 30;
String name = "Alice";
boolean admin = false;
```

Objekte:

```
User user = new User();
```

int, boolean → primitive Typen

String, User → Objekte

Klassen und Methoden:

```
class User {  
    String name;  
  
    String getName() {  
        return name;  
    }  
}
```

Mit Konstruktor:

```
class User {  
    String name;  
  
    User(String name) {  
        this.name = name;  
    }  
}
```

this.name → Feld des aktuellen Objekts

name → Parameter

Arrays und Collections:

```
int[] arr = {1, 2, 3};  
  
arr[0];  
arr.length;
```

Statisches Größe:

```
List<String> users = new ArrayList<>();  
  
users.add("Alice");  
users.get(0);  
users.size();
```

Dynamisch:

```
Map<String, String> user = new HashMap<>();  
  
user.put("name", "Alice");  
user.get("name");
```

Map:

speichert Werte über einen Key statt über einen numerischen Index.

Exceptions:

```
try {  
    readFile();  
} catch (IOException e) {  
    System.out.println(e.getMessage());  
}
```

⇔

```
void read() throws IOException {  
    // ...  
}
```

Kann Fehler verschlucken:

```
catch (Exception e) {  
    // nichts  
}
```

Dateien und Pfade:

```
String filename = "test.txt";  
  
Path path = Paths.get("/uploads", filename);  
  
String content = Files.readString(path);
```

DB:

```
String id = request.getParameter("id");

String sql =
    "SELECT * FROM users WHERE id = " + id;

Statement stmt = connection.createStatement();
stmt.executeQuery(sql);
```

Unsicher:

```
PreparedStatement stmt =
    connection.prepareStatement(
        "SELECT * FROM users WHERE id = ?"
    );

stmt.setString(1, id);
stmt.executeQuery();
```

Sicherer:

Spring-Web-Code lesen:

```
@GetMapping("/user")
public User getUser(
    @RequestParam String id
) {
    return service.findUser(id);
}
```

→ /user?id=123 }

```
@GetMapping("/user/{id}")
public User getUser(
    @PathVariable String id
) {
    return service.findUser(id);
}
```

→ /user/123 . Request Body: }

```
@PostMapping("/user")
public void create(
    @RequestBody User user
) {
    service.save(user);
}
```

@RequestParam, @PathVariable, @RequestBody, @RequestHeader → potentiell User Input

Review:

```
@GetMapping("/download")
public String download(
    @RequestParam String file
) throws IOException {

    return Files.readString(
        Paths.get("/uploads/" + file)
    );
}
```

PowerShell:

PowerShell arbeitet mit Cmdlets, .NET-Klassen und Objekten (statt Text); normale .exe-Programme können ebenfalls gestartet werden.

Cmdlets = Verb-Noun. Beispiel: **Get-Process**

Parameter: Verb-Noun -Parameter Wert. Beispiel: **Get-Process -Name *a*** → Prozesse mit „a“ im Namen.

Positionsparameter: Parametername kann bei passenden Parametern entfallen. Beispiel: **Get-Process *a***

Switch-Parameter: kein Wert nötig. Beispiel: **Get-Process msedge -FileVersionInfo**

Hilfe

Get-Help Get-Process

Get-Help Get-Process -Full

Get-Help Get-Process -Online

Syntax in Get-Help:

[] → optional

<String> → erwarteter Typ

<String[]> → mehrere Werte möglich

Eine Syntaxzeile = ein Parameterset.

Befehle suchen:

Get-Command *process*

Get-Command -Verb Get -Noun *Process*

Wichtige Cmdlets:

Get-Content # Datei lesen

Set-Content # Inhalt ersetzen

Add-Content # Inhalt anhängen

Get-ChildItem # Dateien/Ordner anzeigen

Set-Location # Verzeichnis wechseln

Get-Location # aktueller Pfad

New-Item # erstellen

Remove-Item # löschen

Copy-Item # kopieren

Move-Item # verschieben

Rename-Item # umbenennen
Test-Path # Existenz prüfen

Where-Object # filtern
ForEach-Object # jedes Objekt bearbeiten
Select-Object # Properties auswählen
Sort-Object # sortieren
Measure-Object # zählen/messen

Get-Process / Stop-Process
Get-Service / Start-Service / Stop-Service

Write-Host → direkt im Terminal anzeigen
Write-Output → Objekt/Daten in die Pipeline geben

Variablen: beginnen mit \$

\$x = 5.13
\$name = "Alice"
\$isAdmin = \$true

Strings:

"Hello \$name"
"User: \$(\$user.Name)"

PowerShell bestimmt Typen meist automatisch: `$name.GetType()`

Expliziter Typ: Beispiel: `[int]$zahl = Read-Host "Eine Zahl angeben"`
Falscher Typ → Fehler.

Spezielle Werte: `$true`, `$false`, `$null`

VariablennamenBenennung: PascalCase

Objekte: PowerShell verarbeitet hauptsächlich Objekte statt Text.

Property → Eigenschaft/Wert → `$Objekt.Property`
Method → Aktion → `$Objekt.Method()`
AliasProperty → anderer Name einer Property

Beispiel:

\$Dienst = Get-Service wuauserv

\$Dienst.Status

\$Dienst.Stop()

\$Dienst.Refresh()

Member anzeigen: \$Dienst | Get-Member

Alle Properties + Werte: \$Dienst | Select-Object *

Direkt ohne Variable: (Get-Service wuauserv).Start()

PowerShell-interne Objektinformationen:

\$Dienst.PSObject.Members

\$Dienst.PSObject.TypeNames

Klassen & Vererbung:

Klasse = Bauplan eines Objekts.

Objekt = Instanz einer Klasse.

Vererbung: Unterklasse übernimmt Eigenschaften/Methoden ihrer Basisklasse.

\$Dienst.GetType()

\$Dienst.GetType().BaseType

System.Object = oberste Basisklasse der .NET-Typhierarchie.

Pipelines: | gibt Objekte an den nächsten Befehl weiter.

Get-Process | Where-Object { \$_.CPU -gt 10 } → Prozesse mit CPU > 10

\$_ → aktuelles Pipeline-Objekt.

| Format-List * → alle Properties formatiert anzeigen

Navigieren / Dateien:

Get-ChildItem # dir, ls

Set-Location C:\ # cd, cd ..

Get-Location # pwd

Ordner erstellen: `New-Item -ItemType Directory -Name "Hello PowerShell"`

Programm/Skript aus aktuellem Ordner: `.\programm.exe .\script.ps1`

Datei mit Standardprogramm öffnen: `Invoke-Item .\datei.txt` → Öffne date.txt mit dem Standardprogramm von Windows. Also ähnlich wie ein Doppelklick im Explorer.

PowerShell kann auch andere Provider wie die Registry navigieren:

`cd HKCU:\` → HKCU = HKEY_CURRENT_USER → Registry-Bereich des aktuell angemeldeten Benutzers.

HKCU:\ ist kein echtes Laufwerk auf der Festplatte. PowerShell stellt die Registry so dar, als wäre sie eins mit Ordnern.

`Get-PSDrive` → zeigt alle „Laufwerke“, die PowerShell kennt.

Array: `@(...)`

```
$namen = @("Alice", "Bob", "Eve")  
$namen[0]
```

Hashtables: Key-Value-Paare: `@{...}`

```
$user = @{  
    Name = "Alice"  
    Admin = $true  
}  
  
$user.Name  
$user["Name"]
```

if:

```
if ($age -gt 18 -and $isAdmin) {  
    "Allowed"  
}
```

Wichtige Operatoren:

- `-eq` gleich
- `-ne` ungleich
- `-gt` größer
- `-lt` kleiner
- `-ge` größer/gleich
- `-le` kleiner/gleich
- `-like` Wildcard-Vergleich
- `-match` Regex
- `-and` / `-or` / `-not`

Schleifen:

```
foreach ($name in $namen) {  
    Write-Output $name  
}  
$namen | ForEach-Object {  
    Write-Output $_  
}
```

Pipeline-Variante:

Funktionen: param() definiert Parameter einer Funktion/eines Skripts.

```
function Add-Numbers {  
    param(  
        $a,  
        $b  
    )  
  
    return $a + $b  
}
```

↔ mit typen:

```
function Add-Numbers {  
    param(  
        [int]$a,  
        [int]$b  
    )  
  
    return $a + $b  
}
```

Aufruf:

Add-Numbers 5 10

Pflichtparameter:

```
param(  
    [Parameter(Mandatory)]  
    [string]$FilePath  
)
```

Externe Programme: PowerShell kann normale Programme starten: **ping.exe google.com** oder: **Start-Process "ping.exe"**

PowerShell-Code aus einem String: **Invoke-Expression \$command** ↔ **iex \$command** (kurzform)

Fehlerbehandlung: try → ausführen catch → Fehler behandeln finally → wird immer ausgeführt

```
try {  
    Get-Content "datei.txt" -ErrorAction Stop  
}  
catch {  
    Write-Warning "Fehler: $_"  
}
```

Review:

```
$hostName = Read-Host "Host"
```

```
Invoke-Expression "ping $hostName"
```

```
$file = Read-Host "Filename"
```

```
Invoke-Expression "Get-Content C:\uploads\$file"
```

```
param(  
    [Parameter(Mandatory = $true)]  
    [string]$FilePath  
)  
  
Get-Content $FilePath | ForEach-Object {  
    $command = $_.Trim()  
  
    # Leere Zeilen und Kommentare überspringen  
    if ($command -and -not $command.StartsWith("#")) {  
        Write-Host "> $command"  
        Invoke-Expression $command  
    }  
}
```

.Trim(): entfernt Leerzeichen und ähnliche Whitespace-Zeichen am Anfang und Ende des Textes.

Bash: Useful Cheatsheet: <https://devhints.io/bash>

Always starts with: `#!/bin/bash` To run: `chmod +x yourfile.sh`

then: `./yourfile.sh`

Variables:

`name="Ali"` → Beim Zuweisen kein `$`

`echo $name` → Beim Lesen `$`

Strings: `echo "Hello $name"`

Comment: `#`

Argumente eines Scripts:

```
#!/bin/bash
```

```
echo "$1"
```

```
echo "$2"
```

```
./script.sh Alice 30
```

`$#` → Anzahl Argumente, `$@` → alle Argumente, `$0` → Scriptname

Interactive Input: `read name echo $name`

Arrays:

```
arr=(10 20 30) echo "${arr[0]}"
```

`unset transport[1]` removes bike `transport[1]='train'` sets train.

If: `-eq (=)`, `-ne (!=)`, `-gt (>)`, `-lt (<)`, `-ge (>=)`, `-le (<=)`.

```
if [[ "$name" == "Alice" ]]; then
    echo "yes"
fi
```

Strings:

```
[[ "$a" == "$b" ]]
[[ "$a" != "$b" ]]
```

Zahlen:

```
[[ $a -eq $b ]]
[[ $a -ne $b ]]
[[ $a -gt $b ]]
```

Logik:

```
[[ condition1 && condition2 ]]
[[ condition1 || condition2 ]]
```

Schleifen:

```
for file in "${files[@]"; do
    echo "$file"
done
```

Funktionen:

```
greet() {
    echo "Hello $1"
}
greet "Alice"
```

Innerhalb einer Funktion ist `$1` das erste Funktionsargument.

Command Substitution und Pipes:

```
user=$(whoami)
```

`whoami` ausführen → Output in `$user` speichern.

```
command="$1"
```

`eval "$command"` eval interpretiert einen String als Shellcode → mögliche Command Injection.

`command > file.txt` → stdout überschreiben

`command >> file.txt` → stdout anhängen

`command 2> file.txt` → stderr überschreiben

C:

- Alles muss in Funktionen geschehen!
- Nach den Operationen oder Variablen kommt ein ; am Ende.
- Wenn eine Variable zum 2. Mal genutzt wird, dann keine char/int davor wie beim ersten!
- Nur int / void main() wird aufgerufen.

Typen:

```
int age = 30;
char grade = 'A';
float price = 9.5f;
double pi = 3.14;
```

Strings sind char-Arrays:

```
char name[] = "Alice";
```

⇔ A l i c e \0

Pointer:

```
int x = 5;
int *p = &x;
```

&x → Adresse von x

p → speichert Adresse

*p → Wert an dieser Adresse

*p = 10; ⇔ x = 10;

Funktionen:

```
int add(int a, int b) {
    return a + b;
}
```

```
void setValue(int *x) {
    *x = 10;
}
```

```
int n = 5;
```

```
setValue(&n);
```

Pointer als Parameter:

Aufruf:

Stack und Heap:

```
int x = 5;
char buf[100];
```

liegt typischerweise auf dem Stack.

Dynamischer Speicher: `int *p = malloc(sizeof(int));` liegt auf dem Heap. Dann: `free(p);`

Structs:

```
struct User {  
    int id;  
    char name[50];  
};
```

Benutzung:

```
struct User user;  
  
user.id = 1;
```

```
struct User *user;
```

Benutzung:

```
user->id
```

user.id → normales struct

user->id → Pointer auf struct

Typische Sinks:

```
char buf[16];
```

```
strcpy(buf, input);
```

```
, sprintf(buf, "%s", input);
```

```
, printf(input);
```

besser:

```
printf("%s", input);
```

strcpy() → Buffer Overflow prüfen

strcat() → Buffer Overflow prüfen

sprintf() → Buffer Overflow prüfen

printf(input) → Format String Bug prüfen

malloc/free → Lifetime prüfen

arr[index] → Bounds prüfen

pointer → NULL / dangling pointer prüfen

Review:

```
void login(char *username) {  
    char buffer[16];  
  
    strcpy(buffer, username);  
  
    printf("User: %s\n", buffer);  
}
```

HTML/CSS:

file:///Users/Said/Desktop/Python/site.html to exercise

css to make graphics better. 2 Types:

Internal Bsp: `<li style="color:crimson">Ok</li>`

External with new document using: style.css

Secure Code Review:

Code mostly containing security vulnerabilities:

Crypto implementations / usage

Parser, unparser

System configuration (especially changing to live/prod)

Authentication, authorization

Session management

Further security measures (e.g. request throttling)

 (self-developed code, only used in that one software)

```
public string UploadProfilePicture(IFormFile uploadedFile)
{
    // Generate path to save the uploaded file at
    var path = $"./uploads/avatars/{request.User.Id}/{uploadedFile.FileName}";

    // Save the file
    var localFile = File.OpenWrite(path);
    localFile.Write(uploadedFile.ReadToEnd());
    localFile.Flush();
    localFile.Close();

    // Update the profile picture
    UserProfile.UpdateUserProfilePicture(request.User, path)

    return path;
}
```

bsp:

```
// Prevent path traversal by not utilizing the provided file name. Also
// needed to avoid filename conflicts.
var newFileName = GenerateFileName(uploadedFile.FileName);

// Ensure that we did not accidentally save to a folder outside of the base
// folder
if(!canonicalPath.StartsWith(basePath))
{
    return BadRequest("Attempted to save file outside of upload folder");
}

// Ensure only allowed extensions are saved
if(!IsFileAllowedExtension(uploadedAllowedExtensions))
{
    return BadRequest("Extension is not allowed");
}
```

OWASP-style Secure Code Review Flow:

1. Understand the Application: Architecture, components, roles, sensitive data.

Example: Angular Frontend → PHP REST API → MySQL Database

2. Map the Attack Surface: Endpoints, APIs, uploads, background jobs.

Example: POST /login, GET /users/{id}, POST /admin/create-user, POST /upload, GET /download...

3. Identify Sources: Find user-controlled input.

Example: PHP: \$_GET['id'] \$_POST['username'] \$_COOKIE['session'] \$_FILES['file'] Python: request.???.get("id").

4. Trace Data Flow: Follow input through functions and components.

Example: Source → Processing → Security Controls → Sink. HTTP Request → Controller → Service → Repository → DB

5. Identify Sinks: Database, HTML/DOM, OS, filesystem, network requests, deserialization, redirects.

Example: mysqli_query(...), echo \$input; exec(...), include(\$file); fetch(url), unserialize(...), header("Location: " . \$url); **Question:** Can user input reach this sink?

6. Review Security Controls: Source → Sink is **not automatically a vulnerability**.

Check for: Input validation, Output encoding, Sanitization, Authentication, Authorization, Parameterized queries, Safe framework APIs, Secure error handling.

8. Check Reachability: Can an attacker actually reach and control the vulnerable path?

9. Assess Impact: What can the attacker achieve?

10. Document the Finding: Evidence, impact, remediation and verification.

Parallel Checks:

A. Review Business Logic: Check workflows, privileges and security assumptions. Skip expected workflow-steps.

B. Review Secrets and Configuration: **Search:** password, secret, token, api_key, private_key, connectionString, debug etc. **Example:** \$dbPassword = "SuperSecret123";

Then check: Hardcoded secrets? Secrets committed to Git? Environment variables? Secret manager? Debug mode enabled? Insecure default credentials? Excessive permissions?

C. Review Cryptography: **cryptography correctly implemented?** Weak algorithms, Static enc keys, Insecure RNG, Passwords being encrypted instead of hashed, Keys stored next to encrypted data, Incorrect TLS/certificate handling, etc.

D. Review Logging and Error Handling: **Check what gets logged:** No sensitive information, only important security events.

Check whether errors expose: Stack traces, DB information, Internal file paths, Secrets, Internal system architecture.

E. Review Dependencies: Known vulnerable dependency? Outdated library?

Summary:

The shortest version: Attack Surface → Sources → Data Flow → Sinks → Controls → Reachability → Impact.

And in parallel, always review: Authentication → Authorization → Business Logic and other parallel checks.

Baseline Review: Review a large part or all of the application.

Diff-Based Review: Review only changed code.

OWASP Top 10 → Security awareness / major vulnerability categories.

OWASP ASVS → Detailed security requirements that can be used as a review and verification checklist.

For serious code review: Data-flow analysis + business logic review + ASVS

rather than: “I will simply check the OWASP Top 10 one by one.”

Secure functionalities:

Secure Login: the password should be hashed and stored in the database along the username.

Sicher → Argon2 / bcrypt / scrypt / PBKDF2. Unsicher → MD5 / SHA1 / einfacher SHA256 / Verschlüsselung.

from argon2 import PasswordHasher

Sicher: `ph = PasswordHasher()
password_hash = ph.hash(password)`

Prüfen: `ph.verify(password_hash, password)`

Authorization prüfen:

```
app.get("/users/:id", (req, res) => {  
  const user = db.getUser(req.params.id);  
  res.json(user);  
});
```

Es fehlt:

```
if (req.user.id !== req.params.id) {  
  return res.status(403).send("Forbidden");  
}
```

```
if (request.getParameter("role").equals("admin")) {  
  deleteUser();  
}
```

Benutzer bestimmt selbst seine Rolle. Sicherer:

```
User user = authenticatedUser();  
  
if (!user.hasRole("ADMIN")) {  
  throw new AccessDeniedException("Forbidden");  
}  
  
deleteUser();
```

Session Cookies: Unsicher:

```
res.cookie("session", token);
```

Sicher:

```
res.cookie("session", token, {  
  httpOnly: true,  
  secure: true,  
  sameSite: "lax"  
});
```

JWT-Validierung:

```
const payload = jwt.decode(token);  
  
if (payload.admin) {  
  allowAdmin();  
}
```

```
const payload = jwt.verify(  
  token,  
  publicKey,  
  {  
    algorithms: ["RS256"],  
    issuer: "https://auth.example.com",  
    audience: "my-api"  
  })
```

Beim Review: Signatur, Algorithmus, Issuer, Audience, Expiration und auf Angriffe prüfen.

Unsichere temporäre Dateien:

```
char *filename = "/tmp/myapp.tmp";  
  
FILE *f = fopen(filename, "w");
```

Sicherer:

```
char filename[] = "/tmp/myappXXXXXX";  
  
int fd = mkstemp(filename);
```

`mkstemp()` erzeugt sicher eine neue Datei.

Secure against typical attacks:

SQLi: `$db->query("SELECT * FROM users WHERE id = " . $id);`

```
// unsicher  
$stmt = $db->prepare("SELECT * FROM users WHERE id = ?");  
$stmt->execute([$id]);  
  
// sicherer  
$stmt = $db->prepare("SELECT * FROM users WHERE id = ?");  
$stmt->execute([$id]);
```

```
app.get("/documents/:id", async (req, res) => {
  const doc = await db.getDocument(req.params.id);

  if (!doc || doc.ownerId !== req.user.id) {
    return res.status(403).send("Forbidden");
  }

  res.json(doc);
});
```

Noch besser kann die Berechtigung direkt Teil der Query sein:

```
const doc = await db.getDocument({
  id: req.params.id,
  ownerId: req.user.id
});
```

```
const base = path.resolve("/uploads");
const target = path.resolve(base, filename);

if (!target.startsWith(base + path.sep)) {
  throw new Error("Invalid path");
}

fs.readFileSync(target);
```

Path Traversal: `// unsicher`
`fs.readFileSync("/uploads/" + filename);`

XSS prüfen: `const name = req.query.name;`
`res.send(`<h1>Hello ${name}</h1>`);` Sicherer: `const escapeHtml = require("escape-html");`
`res.send(`<h1>Hello ${escapeHtml(name)}</h1>`);`

`const name = location.hash.substring(1);`
`document.getElementById("output").innerHTML = name;` Sicherer: `document.getElementById("output").textContent = name;` textContent → behandelt Inhalt als Text

User Input → HTML → Browser

SSRF prüfen: `url = request.args["url"]`
`response = requests.get(url)` Sicherer: `from urllib.parse import urlparse`
`url = request.args["url"]`
`parsed = urlparse(url)`
`allowed_hosts = {"api.example.com"}`
`if parsed.scheme != "https" or parsed.hostname not in allowed_hosts:`
`return "Invalid URL", 400`
`response = requests.get(url)`

User Input → URL → Server macht Request

```
const next = req.query.next;
```

Open Redirect: `res.redirect(next);`

Sicherer:

```
const allowed = ["/home", "/profile", "/dashboard"];

if (!allowed.includes(req.query.next)) {
  return res.status(400).send("Invalid redirect");
}
```

```
res.redirect(req.query.next);
```

→ Nur interne Ziele erlauben.

Code Injection: eval(input), exec(input), Invoke-Expression \$input

Command Injection: shell=False setzen!

Unsichere Deserialisierung: `obj = pickle.loads(request.data)`

Sicherer: `obj = json.loads(request.data)` +

```
if not isinstance(obj, dict):
    raise ValueError("Invalid input")
```

Untrusted data → deserialize → Object creation / code paths. reines Datenformat wie JSON + Struktur Validation.

```
$file = $_FILES["file"];

move_uploaded_file(
    $file["tmp_name"],
    "/uploads/" . $file["name"]
);
```

File Upload:

Sicherer:

```
$tmp = $_FILES["file"]["tmp_name"];

$type = mime_content_type($tmp);

$allowed = [
    "image/jpeg" => "jpg",
    "image/png" => "png"
];

if (!isset($allowed[$type])) {
    exit("Invalid file");
}

$name = bin2hex(random_bytes(16)) . "." . $allowed[$type];

move_uploaded_file(
    $tmp,
    "/safe-storage/" . $name
);
```

- Typ prüfen
- eigenen Dateinamen erzeugen
- Größenlimit
- außerhalb ausführbarer Verzeichnisse speichern.

Hardcoded Secrets:

```
DB_PASSWORD = "SuperSecret123"
API_KEY = "abc123"
```

Sicherer:

```
import os

DB_PASSWORD = os.environ["DB_PASSWORD"]
API_KEY = os.environ["API_KEY"]
```

Secret im Source Code → Git History / Logs / Builds / Entwicklerzugriff. Über Secret Management bzw. sichere Konfiguration.

Crypto und Randomness:

```
Random random = new Random();
int token = random.nextInt();
```

Wenn das ein Security-Token sein soll. Oder schwaches Hashing-Alg.

Information Disclosure:

Sicherer: `logger.info("Login attempt for user {}", username);`

Sensitive Data / Logging / Errors. Bsp: `logger.info("Login password: " + password);`

```
app.post("/users/:id", (req, res) => {
  const user = db.getUser(req.params.id);

  if (user.id !== req.user.id) {
    return res.status(403).send("Forbidden");
  }

  const { name, email } = req.body;

  user.name = name;
  user.email = email;

  db.save(user);
  res.json(user);
});
```

IDOR / Mass Assignment:

Authorization:

```
user.id === req.user.id
```

Mass Assignment:

nur erlaubte Felder übernehmen
→ name, email

CSRF:

```
if ($_SERVER["REQUEST_METHOD"] === "POST") {
  deleteAccount($_SESSION["user_id"]);
}
```

```
if (!hash_equals($_SESSION["csrf_token"], $_POST["csrf_token"])) {
  http_response_code(403);
  exit;
}

deleteAccount($_SESSION["user_id"]);
```

State-changing Request + Cookie-Auth → CSRF-Schutz prüfen

Format String:

Unsicher: `printf(user_input);`

Sicherer:

`printf("%s", user_input);`

Übung:

```
from flask import request
import subprocess

@app.route("/convert")
def convert():
    filename = request.args.get("file")
    output = request.args.get("output")

    if not filename:
        return "Missing file", 400

    command = f"convert /uploads/{filename} /tmp/{output}"

    subprocess.run(command, shell=True)

    return "done"
```

Sichern:

```
from pathlib import Path
import subprocess

base_in = Path("/uploads").resolve()
base_out = Path("/tmp").resolve()

src = (base_in / filename).resolve()
dst = (base_out / output).resolve()

if base_in not in src.parents:
    return "Invalid input path", 400

if base_out not in dst.parents:
    return "Invalid output path", 400

subprocess.run(
    ["convert", str(src), str(dst)],
    shell=False,
    check=True
)
```

Erklärung: resolve() normalisiert den Pfad, also z. B.: /uploads/../etc/passwd → /etc/passwd

```
if base_in not in src.parents:
    return "Invalid input path", 400
```

→ verhindert, dass src außerhalb von /uploads liegt.

```
subprocess.run(
    ["convert", str(src), str(dst)],
    shell=False,
    check=True
)
```

["convert", src, dst] → Programm + Argumente getrennt.

check=True → Fehler vom Prozess werden nicht ignoriert.

```
<?php
$id = $_GET["id"];

$sql = "SELECT * FROM users WHERE id = " . $id;

$result = $db->query($sql);

echo $result["email"];
```

Fix:

```
$stmt = $db->prepare(
    "SELECT * FROM users WHERE id = ?"
);

$stmt->execute([$id]);
$result = $stmt->fetch();
```